



<https://mcircuitdesign.com/>

H-DRV8842-R0 DC Motor Sürücü Kartı

1. Konu

1.1 H-DRV8842-R0, DRV8842 entegresi tabanlı, tek H-köprü akım kontrollü bir DC motor sürücü modülüdür.

Kart, 8,2V ile 45V arasında geniş bir besleme gerilimi aralığında çalışarak farklı motor uygulamalarına uyum sağlar.

Kartın en önemli avantajlarından biri 5 bitlik sargı akım kontrolüdür; bu sayede 32 farklı akım seviyesi arasından seçim yapılarak motor hareketi hassas şekilde ayarlanabilir.

Düşük MOSFET RDS(on) değeri (tipik 0,1 Ω , HS+LS) sayesinde güç kayıpları minimize edilir ve verimlilik artırılır.

Kart, 24V ve 25°C koşullarında 5A'e kadar sürücü akımı sağlayabilir.

Endüstri standardı PWM kontrol arayüzü sayesinde birçok mikrodenetleyici ile kolayca entegre edilebilir.

Ayrıca aşırı akım koruması (OCP), termal kapanma (TSD) ve düşük gerilim kilitleme (UVLO) gibi gelişmiş koruma özellikleri sayesinde hem kart hem de bağlı motor güvenli şekilde çalışır.

Hata durumu göstergesi (FAULT) pini ile olası arızalar anlık olarak takip edilebilir.

Kompakt boyutu, sağlam koruma mekanizmaları ve kolay kurulumu ile hem hobi projelerinde hem de endüstriyel uygulamalarda güvenle kullanılabilir.

İşte başlıca özellikleri ve fonksiyonları:

- Tek H-köprü akım kontrollü motor sürücü
- Çalışma besleme gerilimi aralığı: 8,2V - 45V

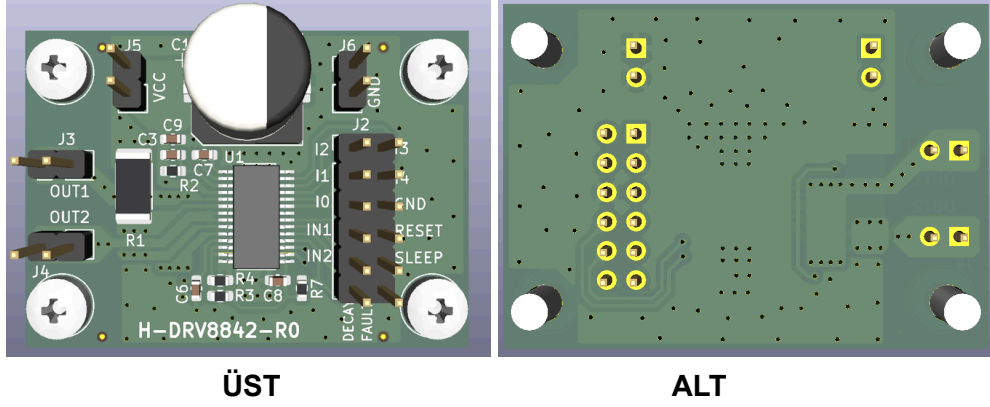
- 5-Bit sargı akım kontrolü (32 farklı akım seviyesi)
- Düşük MOSFET RDS(on): tipik 0,1 Ω (HS + LS)
- 24V, 25°C'de maksimum 5A sürücü akımı
- Endüstri standardı PWM kontrol arayüzü
- Koruma özellikleri:
- Aşırı akım koruması (OCP)
- Termal kapanma (TSD)
- VM düşük gerilim kilitleme (UVLO)
- Hata durumu göstergesi pini (nFAULT)

2. Uygulama

Uygulamamız için gerekli olanlar ; bir **Arduino Uno** , **H-DRV8842-R0 DC Motor Sürücü Kartı** , **DC Motor** ve bileşenlerin bağlantısı için gerekli “**Krokodil kablo**”.



2.1 Pin Dizilimi



IN1	GİRİŞ 1	LOJİK GİRİŞ OUT1'in durumunu kontrol eder. Dahili aşağı çekme (pulldown) direnci vardır.
IN2	GİRİŞ 2	LOJİK GİRİŞ OUT 2'nin durumunu kontrol eder. Dahili aşağı çekme (pulldown) direnci vardır.
I0	AKIM AYAR GİRİŞLERİ	Sarım akımını tam ölçeğin yüzdesi olarak ayarlar. Dahili aşağı çekme direnci vardır
I1		
I2		
I3		
I4		
DECAY	SÖNÜMLEME MODU	Düşük = yavaş sönmleme, açık = karışık sönmleme, yüksek = hızlı sönmleme. Dahili aşağı çekme ve yukarı çekme (pullup) dirençleri vardır.
RESET	SIFIRLAMA GİRİŞİ	Aktif-düşük sıfırlama girdisi, mantığı başlatır ve H-köprüsü çıkışlarını devre dışı bırakır. Dahili aşağı

		çekme direnci vardır.
SLEEP	UYKU MODU GİRİŞİ	Cihazı etkinleştirmek için mantıksal yüksek, düşük güçte uyku moduna girmek için mantıksal düşük. Dahili aşağı çekme direnci vardır.
FAULT	HATA	Hata durumundayken (aşırı sıcaklık, aşırı akım) Lojik düşük
VCC-GND	KÖPRÜ A GÜÇ KAYNAĞI	Motor beslemesine bağlayın (8.2 - 45 V).

2.2 H-DRV8842-R0 DC / Arduino Bağlantı

<u>H-DRV8842-R0</u>	<u>ARDUİNO UNO</u>
<u>IN1 (PWM)</u>	<u>D11</u>
<u>IN2 (PWM)</u>	<u>D3</u>
<u>I0</u>	<u>D4</u>
<u>I1</u>	<u>D5</u>
<u>I2</u>	<u>D6</u>
<u>I3</u>	<u>D7</u>
<u>I4</u>	<u>D8</u>
<u>RESET</u>	<u>D9</u>
<u>SLEEP</u>	<u>D10</u>
<u>VCC-GND</u>	<u>Motor Ayrı Bir Güç ile 8.2V - 45V arası besleme</u>
<u>DECAY</u>	<u>Bu Uygulama için boş bırakıyoruz</u>
<u>FAULT</u>	<u>Bu Uygulama için boş bırakıyoruz</u>

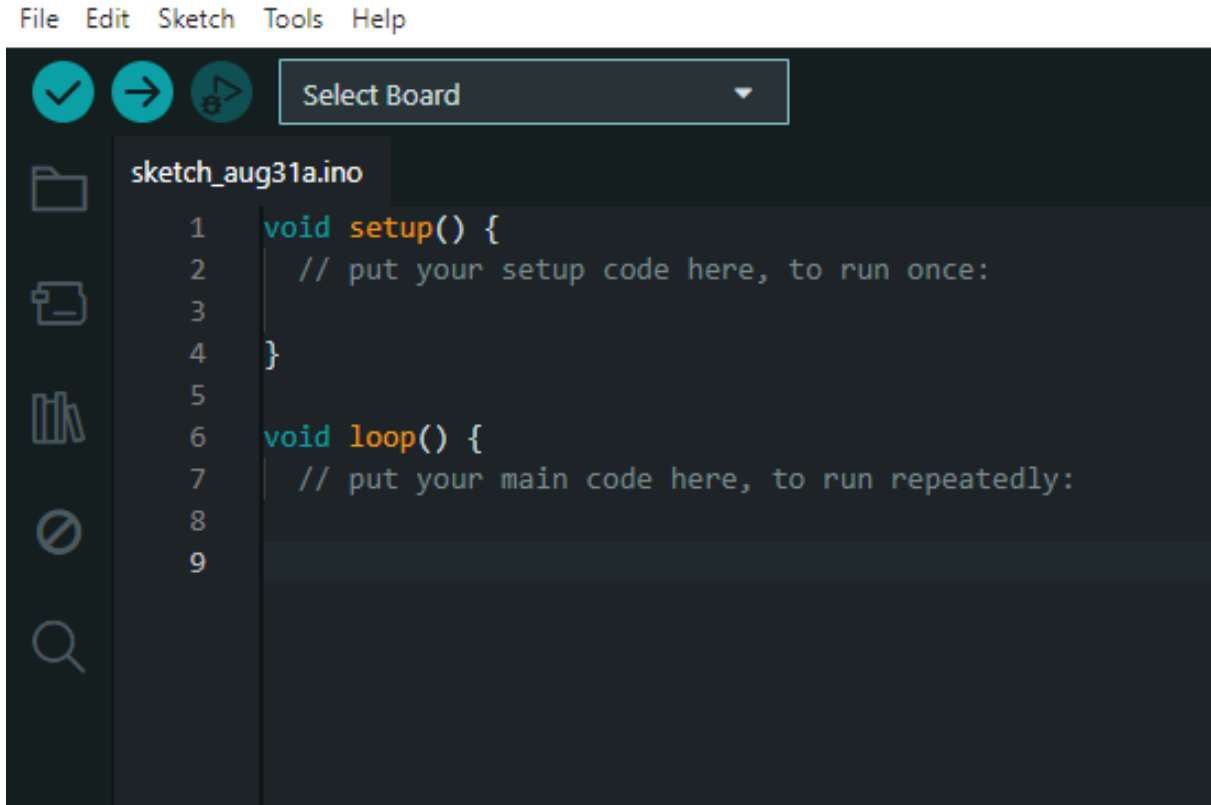
3. Arduino IDE ile Programlama



Arduino Uno'yu programlamak için Arduino IDE yazılımını kullanacağız.

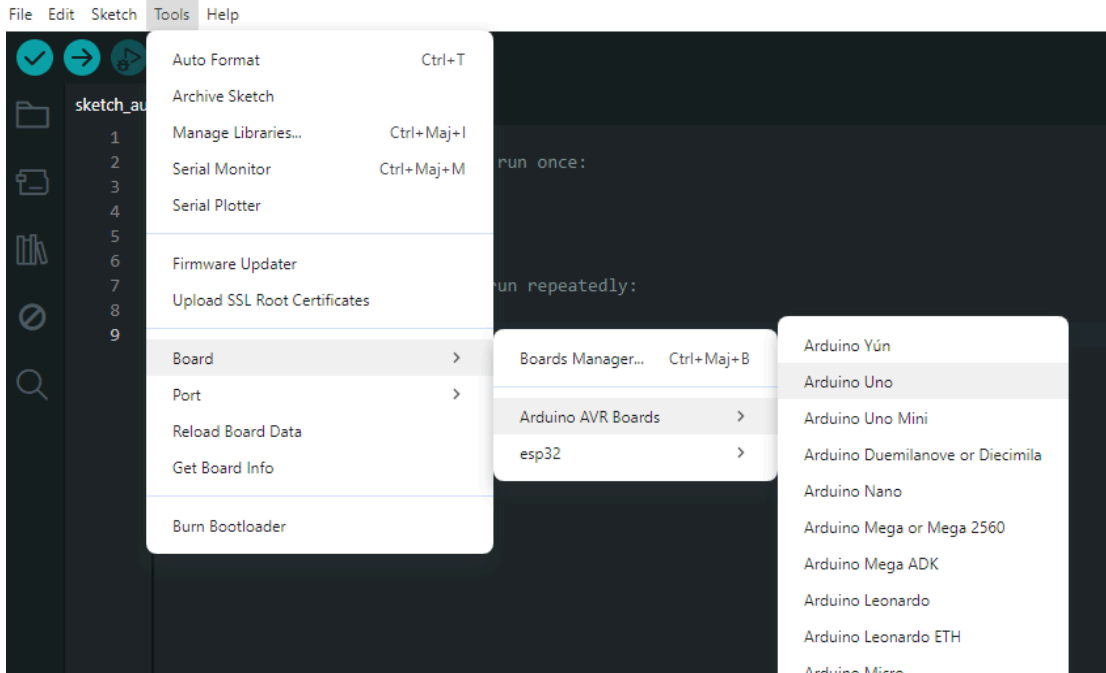
Adım adım ilerleyeceğiz:

1- Arduino IDE'yi açın



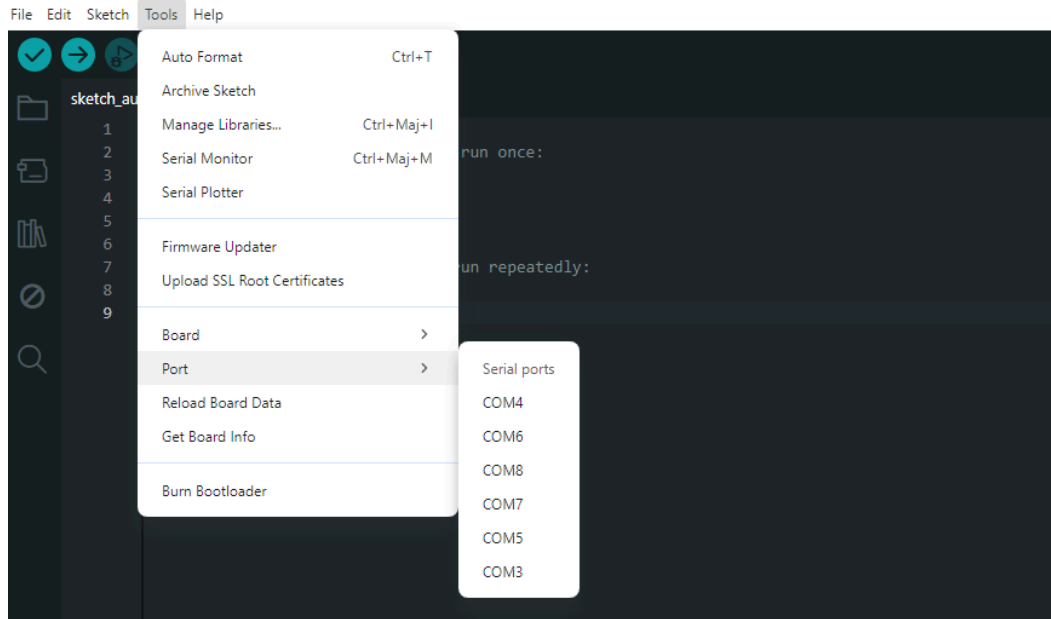
2- Uygun kartı seçin

(Tools > Board > Arduino AVR Boards > Arduino Uno menüsünden "Arduino Uno" seçilir.)



3- Uygun portu seçin

(Tools > Port menüsünden, Arduino Uno'nun bilgisayara bağlı olduğu COM portu (Windows) veya /dev/tty... (Mac/Linux) seçilir.)



4- Programda Pinleri Belirtme

```
const int IN1 = 11, IN2 = 3;  
const int I0 = 4, I1 = 5, I2 = 6, I3 = 7, I4 = 8;  
const int RESET = 9, SLEEP = 10;
```

5-Hız Kontrolü

Hız kontrolü PWM ile gerçekleştirir bu değer 0-255 arasındadır .

```
int motorSpeed = 200; // 0-255 arası PWM değeri (hız)
```

bu uygulama da bu hız 200 olarak belirlendi bu değeri kullanıcı isteğine göre değiştirebilir

6-Seri Haberleşme Başlatma ve Pinlerin giriş/çıkış olarak ayarlanması

Seri haberleşme Arduino ile Bilgisayarınızın Haberleşmesini başlatır

9600 haberleşme hızıdır.

```
void setup() {  
  Serial.begin(9600);  
  
  pinMode(IN1, OUTPUT);  
  pinMode(IN2, OUTPUT);  
  pinMode(I0, OUTPUT);  
  pinMode(I1, OUTPUT);  
  pinMode(I2, OUTPUT);  
  pinMode(I3, OUTPUT);  
  pinMode(I4, OUTPUT);  
  pinMode(RESET, OUTPUT);  
  pinMode(SLEEP, OUTPUT);  
}
```

7-Reset ve Sleep Aktif Hale Getirme

```
digitalWrite(RESET, HIGH);  
digitalWrite(SLEEP, HIGH);
```

8 - Akım seviyesi Ayarı

```
setCurrent(15); // akım seviyesi (0-31)
```

9 - Başlangıcı Serial Monitörde Görmek

Delay(1000) → 1sn eşdeğerdir burda bekleme ekleriz , sebebi motorun veya sürücünün bir komut değişikliğine geçiş yapması için kısa bir süre tanımadır.

```
Serial.println("DRV8842 hiz kontrollu motor testi basliyor...");  
delay(1000);  
}
```

9 - Akım Ayar Fonksiyonu


```
void setCurrent(int level) {  
    digitalWrite(I0, bitRead(level, 0));  
    digitalWrite(I1, bitRead(level, 1));  
    digitalWrite(I2, bitRead(level, 2));  
    digitalWrite(I3, bitRead(level, 3));  
    digitalWrite(I4, bitRead(level, 4));  
}
```

bitRead fonksiyonu, verilen sayının ikilik sistemdeki bitlerini tek tek kontrol eder. Programda akım seviyesi olarak 15 değeri veriliyor. 15 sayısının ikilik karşılığı 01111'dir. Program bu sayının her bir basamağını tek tek okuyarak I0, I1, I2, I3 ve I4 pinlerine gönderir.

Böylece DRV8842 motor sürücüsü, hangi akım seviyesinde çalışması gerektiğini bu pinlerden anlar.

Yani basitçe:

15 sayısı → binary olarak 01111 → bitler tek tek okunur → I0-I4 pinlerine gönderilir → motor sürücüsünün akım seviyesi belirlenir.

10 - Motoru Durdurma Fonksiyonu

```
void motorDurdur() {  
    digitalWrite(IN1, LOW);  
    digitalWrite(IN2, LOW);  
}
```

IN1 ve IN2 Low yani alt konuma getirilir ve motor dönmez

Programın ilerleyen kısmında:

motorDurdur(); satırının bu fonksiyon olduğunu anlarız

10 - Motor ileri ve Geri yani sağ dönme ve sola dönme fonksiyonları.

İleri :

```
void motorIleri(int hiz) {  
    analogWrite(IN1, hiz);  
    digitalWrite(IN2, LOW);  
}
```

*Bu fonksiyon motoru **ileri yönde** döndürmek için kullanılıyor.*

Burada iki şey yapılıyor :

IN1

analogWrite(IN1, hiz);

IN1 pinine PWM uygulanıyor.

örneğin `motorIleri(80);` denirse

IN1 = PWM 80

oluyor.

IN2 kapatılıyor :

```
digitalWrite(IN2, LOW);
```

Sonuç:

```
IN1 = PWM  
IN2 = LOW
```

motor bir yönde dönüyor.

Geri :

```
void motorGeri(int hiz) {  
    digitalWrite(IN1, LOW);  
    analogWrite(IN2, hiz);  
}
```

Burada tam tersi yapılıyor.

```
IN1 = LOW
```

```
IN2 = PWM
```

Böylece motorun yönü değişiyor.

11 - loop() bölümü

Şimdi programın sürekli tekrarlanan kısmına geldik :

loop () Arduino tarafından **sonsuz şekilde tekrar tekrar çalıştırılır.**

Programın motor testi esas olarak burada yapılıyor.

Akış şu:

```
Yavaş ileri
```

```
↓
```

```
2 saniye
```

```
↓
```

```
Hızlı ileri
```

```
↓
```

```
2 saniye
```

```
↓
```

```
Dur
```

```
↓
```

```
1 saniye
```

```
↓
```

```
Yavaş geri
```

```
↓
```

```
2 saniye
```

```
↓
```

```
Hızlı geri
```

```
↓
```

```
2 saniye
```

```
↓
```

```
Dur
```

```
↓
```

```
1 saniye
```

```
↓
```

```
Tekrar başa
```

Programda ise :

```
void loop() {  
  // Yavaş ileri  
  Serial.println("Motor YAVAS ileri (hiz=80)...");  
  motorIleri(80);  
  delay(2000);  
  
  // Hızlı ileri  
  Serial.println("Motor HIZLI ileri (hiz=255)...");  
  motorIleri(255);  
  delay(2000);  
  
  motorDurdur();  
  Serial.println("Motor DURUYOR...");  
  delay(1000);  
  
  // Yavaş geri  
  Serial.println("Motor YAVAS geri (hiz=80)...");  
  motorGeri(80);  
  delay(2000);  
  
  // Hızlı geri  
  Serial.println("Motor HIZLI geri (hiz=255)...");  
  motorGeri(255);  
  delay(2000);  
  
  motorDurdur();  
  Serial.println("Motor DURUYOR...");  
  delay(1000);  
}
```

Bu değerler sizin isteğinize göre değiştirilebilir..

Programın Tam Hali :

```
// DRV8842 Motor Test Programı - Hız Kontrollü

const int IN1 = 11, IN2 = 3;

const int I0 = 4, I1 = 5, I2 = 6, I3 = 7, I4 = 8;

const int RESET = 9, SLEEP = 10;


int motorSpeed = 200; // 0-255 arası PWM değeri (hız)


void setup() {

    Serial.begin(9600);


    pinMode(IN1, OUTPUT);

    pinMode(IN2, OUTPUT);

    pinMode(I0, OUTPUT);

    pinMode(I1, OUTPUT);

    pinMode(I2, OUTPUT);

    pinMode(I3, OUTPUT);

    pinMode(I4, OUTPUT);

    pinMode(RESET, OUTPUT);

    pinMode(SLEEP, OUTPUT);


    digitalWrite(RESET, HIGH);

    digitalWrite(SLEEP, HIGH);


    setCurrent(15); // akım seviyesi (0-31)
```

```
Serial.println("DRV8842 hiz kontrollu motor testi basliyor...");

delay(1000);

}

void setCurrent(int level) {

    digitalWrite(I0, bitRead(level, 0));

    digitalWrite(I1, bitRead(level, 1));

    digitalWrite(I2, bitRead(level, 2));

    digitalWrite(I3, bitRead(level, 3));

    digitalWrite(I4, bitRead(level, 4));

}

void motorDurdur() {

    digitalWrite(IN1, LOW);

    digitalWrite(IN2, LOW);

}

void motorIleri(int hiz) {

    analogWrite(IN1, hiz);

    digitalWrite(IN2, LOW);

}

void motorGeri(int hiz) {

    digitalWrite(IN1, LOW);
```

```
    analogWrite(IN2, hiz);  
}  
  
void loop() {  
    // Yavaş ileri  
    Serial.println("Motor YAVAS ileri (hiz=80)...");  
    motorIleri(80);  
    delay(2000);  
  
    // Hızlı ileri  
    Serial.println("Motor HIZLI ileri (hiz=255)...");  
    motorIleri(255);  
    delay(2000);  
  
    motorDurdur();  
    Serial.println("Motor DURUYOR...");  
    delay(1000);  
  
    // Yavaş geri  
    Serial.println("Motor YAVAS geri (hiz=80)...");  
    motorGeri(80);  
    delay(2000);  
  
    // Hızlı geri  
    Serial.println("Motor HIZLI geri (hiz=255)...");
```

```
motorGeri(255);  
  
delay(2000);  
  
motorDurdur();  
  
Serial.println("Motor DURUYOR...");  
  
delay(1000);  
}
```